

TAKE-HOME TECHNICAL ASSIGNMENT

SENIOR FULL-STACK .NET MODERNIZATION DEVELOPER

1. PURPOSE OF THIS ASSIGNMENT

This take-home assignment is the first phase of our technical interview process. The goal is to understand how you:

- Design and build maintainable applications
- Work with databases in a real-world, production-support mindset
- Apply business constraints such as soft deletion
- Integrate a React user interface, C# API, database, and business logic
- Use AI responsibly while retaining ownership of decisions

This is not an exam. We are not looking for a perfect or production-ready application. We are interested in your thinking, structure, and decision-making.

2. USE OF AI TOOLS

- AI tools (e.g., ChatGPT, Copilot) are allowed.
- You must be able to explain and justify all code and design choices.
- AI-generated content without understanding will be identified during follow-up discussions.

AI is treated as a tool, not a substitute for engineering judgment.

3. SUBMISSION INSTRUCTIONS

The starter code for this assignment is provided through a GitHub repository. Use GitHub to complete and submit your work.

HOW TO COMPLETE THE ASSIGNMENT

- Clone or fork the project using the provided GitHub repository link.
- Create a new branch from the main branch.
- Complete all work in your own branch.
- Do not make changes directly to the main branch; it contains the original starter solution.

HOW TO SUBMIT YOUR WORK

When finished, share one of the following:

- A link to your GitHub repository, if you forked the project
- A link to the specific branch or pull request containing your completed work

You do not need to submit a ZIP file or email the source code.

WHAT YOUR SUBMISSION SHOULD INCLUDE

- Complete source code for the C# API and React application
- The SQLite database file (.db)
- Any SQL scripts used to create or modify tables, views, or queries
- A README.md explaining:
 - How to run the API and React application locally
 - Database design decisions
 - How soft deletion is implemented
 - How the React UI communicates with the C# API
 - Assumptions and trade-offs
 - What you would improve with more time

FINAL CHECK

Your application should:

- Build successfully
- Run locally, with the React UI communicating with the C# API
- Be clearly documented so another developer can follow the setup instructions

4. BUSINESS CONTEXT AND DOMAIN

The system manages Providers and their associated Licenses. This reflects real internal applications supported and maintained by our organization.

Key Organizational Rule: Hard deletion is not allowed. All delete operations must use soft deletion. No soft-delete columns or flags are provided upfront; you are expected to design this behavior.

5. REQUIRED DOMAIN CONCEPTS

PROVIDER

- Unique identifier
- Provider name
- County
- Status or lifecycle state
- Soft-delete behavior
- Audit-safe persistence

LICENSE

- Unique identifier
- Association with a provider
- License number
- License status
- Expiration date

You may extend the model if needed.

6. DATA DICTIONARY REFERENCE

The following data dictionary describes the business meaning of the core entities. You are responsible for designing the physical SQLite schema based on these definitions and the assignment constraints. You may add fields or tables as needed; document them in the README.

PROVIDER

Represents an organization or individual that provides services and is managed by state staff.

- **ProviderId:** Unique identifier and primary key; implementation choice is yours.
- **ProviderName:** Searchable display name.
- **County:** County where the provider operates (e.g., Fulton, DeKalb).
- **Status:** Business or lifecycle status; examples include Active, Inactive, and Pending.
- **CreatedDate:** Record creation date for audit and tracking.

Notes:

- Providers must not be physically deleted.
- Soft-deleted providers must not appear in standard application views.
- Soft-delete design decisions are a key evaluation area.

LICENSE

Represents a license issued to a provider that authorizes operations for a defined period.

- **LicenseId:** Unique identifier and primary key.
- **ProviderId:** Foreign key to the associated provider.
- **LicenseNumber:** Business-recognizable identifier, usually unique per provider.
- **LicenseStatus:** Current status; examples include Active, Expired, and Suspended.
- **ExpirationDate:** Date on which the license expires; used to determine validity.

BUSINESS RULES AND RELATIONSHIPS

- A Provider can have zero or more Licenses.
- A License must belong to one Provider.
- Provider status does not necessarily imply license validity.
- License expiration is determined by ExpirationDate.
- Soft-deleted providers must remain in the database, be excluded from standard views and operations, and remain queryable for audit or troubleshooting.

7. DATABASE REQUIREMENTS

- Use SQLite.
- You may use tables, views, and reusable query patterns.
- Stored procedures are optional; SQLite-appropriate alternatives are acceptable.

SOFT-DELETE EXPECTATIONS

Your design must ensure:

- Soft-deleted records do not appear in standard application results.
- Deleted records remain queryable for audit or maintenance.
- Soft-delete enforcement is consistent across queries.
- The implementation is clearly explained in the README.

8. APPLICATION FUNCTIONAL REQUIREMENTS

The application must include a React UI and a C# API. The React UI must use the API for application data; do not hard-code provider or license data in the frontend.

8.1 PROVIDER LISTING

- Display providers that are not soft-deleted.
- Load provider data from the C# API.
- Explain the database/query approach used to exclude deleted records.

8.2 CREATE PROVIDER

- Add a provider through the C# API.
- Validate input and preserve data integrity and soft-delete rules.

8.3 EDIT PROVIDER

- Update provider information through the C# API.
- Validate input and preserve data integrity.

8.4 DELETE PROVIDER (SOFT DELETE ONLY)

- Implement deletion through the C# API using the chosen soft-delete design.
- Deleted providers must remain in the database and must not appear in normal API or UI listings.
- Hard deletion is not allowed.

8.5 LICENSE INFORMATION

- Provide a way in the React UI to view a provider's associated licenses.
- License data must be retrieved from the C# API.
- The API must preserve the provider/license relationship and must not infer license validity solely from provider status.

8.6 REQUIRED DATA SCENARIOS

The solution must support querying for:

- Active providers and their active licenses
- Providers appearing active but having expired licenses
- Exclusion of soft-deleted providers from standard results

These scenarios may be implemented using database objects, application logic, or a combination. Include sample data or document a repeatable way to populate data that demonstrates each scenario.

8.7 C# API REQUIREMENTS

Implement an ASP.NET Core Web API using C# and expose JSON endpoints for the provider workflows and required license/query scenarios. Routes may follow your preferred REST conventions; the following are examples, not mandatory names:

- GET /api/providers to list non-deleted providers
- GET /api/providers/{id} to retrieve a provider and relevant license information
- POST /api/providers to create a provider
- PUT /api/providers/{id} to update a provider
- DELETE /api/providers/{id} to soft-delete a provider
- Additional endpoint(s) or query parameters for the required license scenarios

The API should perform validation and enforce business rules; they must not exist only in the React UI. Return appropriate success and error responses, and document any API assumptions or design choices.

8.8 REACT UI REQUIREMENTS

Build the core user interface with React (JavaScript or TypeScript).

At minimum, the UI must:

- Display providers loaded from the C# API
- Support create and edit workflows
- Support soft deletion with a clear confirmation step
- Display the selected provider's license information
- Make the required active/expired-license scenarios accessible to the user
- Handle loading, empty, validation, and API error states

UI polish is secondary to clear workflows, correct API integration, and maintainable code.

9. TECHNOLOGY STACK

Required:

- C# and ASP.NET Core Web API (.NET 8 or .NET 10)
- React (JavaScript or TypeScript)
- Entity Framework Core with the SQLite provider
- SQLite

You may choose a suitable project structure, frontend tooling, and lightweight UI or charting libraries. Explain material choices in the README.

10. OPTIONAL ENHANCEMENT: INTERACTIVE DASHBOARD (REACT)

This section is optional and does not replace the required React CRUD and license UI described above.

Create an additional read-only dashboard highlighting provider and license insights. Choose any useful metrics, for example:

- Providers by status (bar or pie chart)
- Licenses per provider (bar chart)

- Expired versus active licenses (pie chart)
- Summary cards with key counts
- Providers with expiring licenses

Guidelines:

- Use React; any setup is acceptable.
- Charts may use any lightweight library.
- Data must come from the C# backend, not hard-coded values.
- Business logic must not exist only in the UI.

If implemented, explain why you chose the metrics and how the frontend and backend are integrated.

11. SOLUTION STRUCTURE

You are free to structure the solution based on your experience and best practices. No specific folder structure is required. Be prepared to explain your organizational choices.

12. WHAT WE EVALUATE

We evaluate submissions based on:

- Application architecture and clarity
- React-to-API integration and separation of responsibilities
- Database design and reasoning
- Quality and consistency of soft-delete implementation
- Backend and database integration
- Maintainability and production-support awareness
- Documentation quality
- Optional dashboard depth, if implemented

UI polish and advanced styling are not primary criteria.

13. WHAT HAPPENS NEXT

- Submission review
- Technical discussion and Q&A
- On-site, hands-on interview for finalists
- Final hiring decision

14. FINAL NOTES

- Partial completion is acceptable.
- Clear reasoning is more important than completeness.
- This assignment reflects real-world maintenance work.

Thank you for your time.

SUBMISSION DEADLINE

Please complete and submit your assignment as soon as you are able. If a specific submission deadline is communicated separately, ensure your submission is completed by that date. Otherwise, no fixed deadline applies.